Fast cross-attention backward for few-query / many-key transformer decoders on GPUs

Team Members: Raymond He, Atindrah Harishankar

URL

https://atindrah-hari.github.io/15418-Final-Project-Attention-Kernel/

SUMMARY

We will implement and optimize a custom CUDA kernel for the backward pass of the masked "queries attend to nodes" cross-attention in a MaskFormer-style transformer, where 512 to 1536 queries attend to 32k to 65k nodes with a head dimension of 8. FlashAttention cannot be used for this layer because it does not support our dynamic attention masks (it is used only in the encoder), so the decoder runs PyTorch's memory-efficient attention kernel. Our baselines are PyTorch's memory efficient kernel and unfused math attention, which we will compare across different problem sizes and across several GPUs (RTX 4090, PSC GPUs, and a research-lab GPU), analyzing what limits performance on each.

BACKGROUND

The application is a MaskFormer-style transformer for particle reconstruction from calorimeter hits. An encoder processes tens of thousands of hit nodes per event (can go up to hundreds of thousands), and a decoder with a few hundred/thousand learned queries repeatedly attends to those hits. Each decoder layer has a cross-attention in which queries attend to nodes, using an attention mask that restricts which nodes each query can see. The mask is data-dependent, meaning each decoder layer uses the mask predicted by the previous layer, so it changes with every layer and event and thus cannot be precomputed. The current model dimension is 128 with 16 heads, so the head dimension is 8.

The backward pass of attention computes the gradients of every query, key, and value (the gradients dQ, dK and dV). In this layer, there are only a few hundred queries but tens of thousands of keys. Each query is only allowed to attend to the keys selected by its mask, and masked pairs contribute nothing to the gradients. A key's gradient is therefore a short sum over the queries that selected it, while a query's gradient is a long sum over all the keys it selected. Different keys, heads and batch elements are independent, so a GPU can process them in parallel. However, splitting the keys across many threads means they all add into the same few query results, which needs careful coordination, and the kernel must find which query-key pairs are active, since the mask is only known at runtime.

The encoder uses FlashAttention. However, FlashAttention does not support our arbitrary attention masks, so the decoder uses PyTorch's memory-efficient scaled-dot-product attention, which supports these masks. PyTorch's unfused math attention is the other path that supports

arbitrary masks, but it materializes the full Q × N score matrix and so is memory-hungry at our sizes (causes paging from main memory). The memory-efficient kernel is designed for general attention shapes, where the numbers of queries and keys are similar and the head dimension is typically 64 to 128, so we expect a kernel specialized for our shape, mask and head dimension to do better.

We have two real configurations to target: 512 queries by 32,768 padded nodes at at least batch size 4 (about 21k real hits per event), and 1,536 queries by 65,536 padded nodes at batch size 1 (about 34k real hits per event). That is a key-to-query ratio of about 64:1 and 43:1. Real data can reach up to about 200k hits per event, and later runs will also include pileup, so we will also scale to much larger events to see how the kernel behaves as the problem grows.

THE CHALLENGE

- **Workload shape:** as noted above, with 43 to 64 times more keys than queries and a head dimension of only 8, the arithmetic per key is tiny (high communication-to-computation ratio), so the kernel is likely limited by memory traffic and by the reduction structure rather than by math. It is unclear whether parallelizing over keys or over queries is better, and by how much.
- **Dependencies and reduction:** dQ reduces over keys while dK and dV reduce over queries. Parallelizing over either dimension makes one set of gradients easier to compute, but introduces synchronization or additional reduction steps for the other.
- **Masking and irregularity:** FlashAttention does not support our arbitrary masks. Masks also make some tiles empty or partially filled, causing divergence and wasted work. Additionally, nodes are padded to a fixed maximum while real events fill only about 65% of it, so handling the padding affects utilization. The mask is recomputed every layer, so the kernel cannot rely on a fixed sparsity pattern.
- **Hardware constraints:** Since we are limited by memory, head dimension will generally be small (currently 8), which gives very little work per key and is below what library kernels are tuned for. Thus, memory bandwidth, shared-memory capacity, register pressure and occupancy dominate. These differ across GPU generations, so the best tile sizes should differ per GPU.
- **What we hope to learn:** which resource limits this shape on each GPU, whether a shape-specialized kernel can beat a general library kernel, and how the best design changes across architectures.

RESOURCES

- **Hardware:** RTX 4090 (local), research-lab GPU (2080 Ti), (optional) PSC GPU nodes (H100 or A100 if possible).
- **Starting code:** the existing PyTorch MaskFormer training code, used only to extract realistic tensor shapes and masks, and PyTorch SDPA with a mask (memory-efficient and math backends) as the baselines. The custom kernel will be written from scratch in CUDA.

- **References:** Dao et al., FlashAttention (2022) and FlashAttention-2 (2023); NVIDIA CUDA programming and profiling documentation.
- **Codebase:** https://github.com/raymondhe212006/MaskFormer-combined

GOALS AND DELIVERABLES

Plan to achieve:

1. A standalone benchmark that tests the cross-attention backward pass using the two real workload configurations, without running the full model.
2. A correct CUDA backward kernel for this shape, checked numerically against a PyTorch reference.
3. A measured comparison against PyTorch's memory-efficient and math masked attention backward across a sweep of Q, N, head dim and mask density on each GPU type.
4. A bottleneck analysis supported by profiler counters (memory bandwidth, occupancy, reduction or atomics cost) that explains the limits of each implementation.

Hope to achieve:

1. A speedup over PyTorch's masked backward kernels on the real shape. We do not yet know the achievable factor, which is part of what we want to measure.
2. Evaluating the kernel inside a real training step to measure performance gains.

If work goes slower:
A working kernel plus an analysis of why the library kernel is slow at this shape, compared across GPUs.

Poster:
Graphs of backward time versus problem size for each implementation and GPU, a breakdown of the limiting factors, and a table of the best configuration per GPU. No live demo planned.

PLATFORM CHOICE

We decided to use GPUs because we needed a high memory bandwidth to fit the backwards attention over the thousands of keys. This would also allow us to parallelize across data, which fits the nature of our input. We plan on testing across different GPUs (4090, PSC, lab machine) to further understand how our design depends and performs across different shared memory, bandwidth and core limitations. We also decided to use CUDA instead of other API's as it gives control over the shared memory and is known to work well for problems with inherent data parallelism.

SCHEDULE

- **Week of Nov 9:** build the benchmarks simulating our predicted shapes and masks. Get baseline timings of each benchmark across machines (4090, PSC, lab machine)
- **Week of Nov 16:** Get an initial correct CUDA backward kernel working. Compare performance against the benchmarks across each machine. Write the milestone report. (This can also be work for the previous week).
- **Week Nov 23:** Optimizations. Our initial ideas include testing different tile sizes, partitioning key-axis, testing different reduction strategies. Compare performance across benchmarks and initial version, analyzing speedup factors.
- **Week of Nov 30:** Run sweeps across the different machines and continue testing optimizations. Profile the program to find bottleneck and try to speed it up
- **Week of Dec 7:** Finalize experiments and report. Create poster and prepare for poster session.
- **Dec 11:** Poster session